

Project Specification

Title: News Article Categorization & Extractive Summarization

Objective: Build an end-to-end system that classifies news articles into categories and generates extractive summaries, with proper MLOps integration and deployment.

Step-by-Step Plan

Step 1. Problem Understanding & Scope

- Task 1: **Classification** — categorize news articles into topics (politics, tech, sports, etc.).
 - Task 2: **Summarization** — generate short summaries using extractive methods.
 - Scope includes **data preprocessing, ML + DL models, pipeline automation, experiment tracking, containerization, and deployment.**
-

Step 2. Dataset Acquisition

- **Datasets:**
 - **AG News Dataset** (news classification into 4 classes).
 - **CNN/Daily Mail Dataset** (longer articles for extractive summarization).
 - Students will download datasets, inspect structure, and merge preprocessing pipelines.
-

Step 3. Data Preprocessing & Cleaning

- Tokenization, stopwords removal, stemming/lemmatization.
 - Convert text into numeric features:
 - **For baseline models:** Bag-of-Words, TF-IDF.
 - **For DL models:** word embeddings (Word2Vec/GloVe).
 - Split into train/validation/test sets.
-

Step 4. Baseline Models (Traditional ML)

- Implement and evaluate:

- **Naive Bayes** (MultinomialNB).
 - **Logistic Regression / SVM**.
 - Evaluate using **accuracy, precision, recall, F1-score**.
 - Record experiments in **MLflow**.
-

Step 5. Deep Learning Models (ANN, CNN, RNN)

- Build text classifiers using:
 - **ANN** on embeddings.
 - **CNN** for text classification.
 - **RNN/LSTM** for sequential text classification.
 - Compare with baseline models.
 - Log training runs, metrics, confusion matrices in **MLflow**.
-

Step 6. Extractive Summarization Module

- Implement summarization methods:
 - **TextRank algorithm** (graph-based ranking of sentences).
 - Optionally compare with simple frequency-based extractive summarizer.
 - Evaluate using **ROUGE scores**.
-

Step 7. Integration of Classification + Summarization

- Build a pipeline where:
 1. A news article is input.
 2. The system classifies it into a category.
 3. Generates a summary (extractive).
 - Save final outputs in structured format (JSON with category + summary).
-

Step 8. Orchestration with Airflow

- Define **Airflow DAG** with tasks:

1. Fetch & preprocess dataset.
 2. Train baseline models.
 3. Train CNN/RNN models.
 4. Generate summaries.
 5. Store results + metrics.
- Schedule retraining periodically (e.g., weekly).
-

Step 9. Containerization & Deployment

- **Docker:** Containerize preprocessing, training, and inference services.
 - **Kubernetes:** Deploy on a cluster with:
 - Classification service (REST API).
 - Summarization service (REST API).
 - Optional: Monitoring via MLflow UI & logs.
 - Create a **simple API/UI**: input an article → output category + summary.
-

Step 10. Testing, Documentation & Final Report

- Perform end-to-end testing on unseen news articles.
 - Document:
 - Dataset details.
 - Model comparisons (traditional ML vs CNN/RNN).
 - Summarization performance.
 - Airflow DAG screenshots.
 - Docker/K8s deployment steps.
 - Write **final report + presentation deck**.
-

Deliverables

1. **Project Report** (PDF/Doc)
 - Problem statement, dataset, methodology, results, and conclusion.

2. **Codebase** (GitHub repo)

- data_preprocessing/
- models/ (ML + DL)
- summarization/
- airflow_dag/
- docker/
- kubernetes/

3. **MLflow Logs & Screenshots** (showing tracked runs).

4. **Airflow DAGs & Execution Logs** (screenshot + .py file).

5. **Docker Images** (for training + inference).

6. **Kubernetes Deployment Files** (.yaml configs).

7. **REST API / Simple UI** (article input → category + summary output).

8. **Presentation Slides** (overview of approach, models, results, deployment).